Vectors

Section Overview

- Vector
- Vector implements a dynamic array. It is similar to ArrayList, but with two differences
- Vector is synchronized.
- Vector contains many legacy methods that are not part of the collections framework.
- Legacy class

A legacy class is a class that was part of Java from its earliest versions (JDK 1.0/1.1, before the Collections Framework existed in JDK 1.2) and was later retrofitted to fit into the newer framework, rather than being designed as part of it from the start.

Section Overview

Synchronized by default: every method locks, even in single-threaded contexts where you're paying for thread-safety you don't need (Vector, Hashtable)

- Predates generics: originally worked with raw Object types; generics were introduced on in JDK 1.5

Has its own parallel API: e.g., Vector has both old-style methods (addElement(), elementAt(), removeElement()) and new List-style methods (add(), get(), remove()) doing the same thing

Uses Enumeration for traversal: Enumeration is read-only (no remove() mid-iteration) and has clunkier method names (hasMoreElements()/nextElement() vs hasNext()/next())

Vector declaration

- 1. Vector()
- Vector<String> v = new Vector<>();
- 2. Vector(int initialCapacity)
- Vector<String> v = new Vector<>(50);
- Pre-allocates space for 50 elements without immediate resizing. Still doubles once that capacity is exceeded.
- 3. Vector(int initialCapacity, int capacityIncrement)
- Vector<String> v = new Vector<>(10, 5);

Once you set capacityIncrement, the growth policy changes from doubling to linear addition: capacity goes 10 → 15 → 20 → 25... instead of 10 → 20 → 40.

- 4. Vector(Collection<? extends E> c)
- List<String> names = Arrays.asList("A", "B", "C");
- Vector<String> v = new Vector<>(names);

Section Overview

- Adding elements
- Vector<String> v = new Vector<>();
- v.addElement(“John"); // legacy style
- v.add("Priya"); // List-style
- v.add(0, "Kumar"); // insert at index
- v.insertElementAt(“Rahul", 1); // legacy style, insert at index

Section Overview

- Accessing elements

```
> System.out.println(v.elementAt(0)); // legacy: “Kumar"
> System.out.println(v.get(0)); // List-style: “Kumar"
> System.out.println(v.firstElement()); // "Kumar"
> System.out.println(v.lastElement()); // "Priya"
```

Section Overview

- Removing elements
- v.removeElement(“John"); // legacy, remove by value
- v.remove("Priya"); // List-style, remove by value
- v.removeElementAt(0); // legacy, remove by index
- v.remove(0); // List-style, remove by index
- v.removeAllElements(); // legacy, clear everything
- v.clear(); // List-style, clear everything

Section Overview

- Searching
- Vector<String> v = new Vector<>();
- v.add("A"); v.add("B"); v.add("C"); v.add("B");

```
> System.out.println(v.contains("B")); // true
> System.out.println(v.indexOf("B")); // 1 (first occurrence)
> System.out.println(v.lastIndexOf("B")); // 3 (last occurrence)
> System.out.println(v.indexOf("B", 2)); // 3 (search starting from index 2)
```

Section Overview

- Size and capacity
- Vector<Integer> v = new Vector<>(5, 2); // initial cap 5, increment 2

```
> System.out.println(v.size()); // 0 (no elements yet)
> System.out.println(v.capacity()); // 5 (allocated space)
```

- for (int i = 0; i < 6; i++) v.add(i);

```
> System.out.println(v.size()); // 6
> System.out.println(v.capacity()); // 7 (5 + increment 2, once exceeded)
```

- v.setSize(3); // truncates to first 3 elements

```
> System.out.println(v); // [0, 1, 2]
```

Section Overview

- Copying and converting
- Vector<String> v = new Vector<>();
- v.add("X"); v.add("Y");
- Vector<String> copy = (Vector<String>) v.clone(); // shallow copy
- Object[] arr = v.toArray(); // to Object[]
- String[] strArr = v.toArray(new String[0]); // to String[]

Section Overview

- Replacing / updating
- Vector<String> v = new Vector<>();
- v.add("X"); v.add("Y");
- v.setElementAt("Z", 0); // legacy: v = [Z, Y]
- v.set(1, "W"); // List-style: v = [Z, W]

Iteration

- Vector<String> v = new Vector<>();
- v.add("A"); v.add("B"); v.add("C");

```
> // Modern: Iterator
```

- Iterator<String> it = v.iterator();
- while (it.hasNext()) {

```
> System.out.println(it.next());
```

- }

```
> // Modern: for-each
```

- for (String s : v) {

```
> System.out.println(s);
```

- }

Emptiness check

- Vector<String> v = new Vector<>();

```
> System.out.println(v.isEmpty()); // true
```

- v.add("A");

```
> System.out.println(v.isEmpty()); // false
```

Iterable and Iterator

- Before these interfaces, every collection type had its own way of being traversed.

Arrays use an index and length, Vector used Enumeration, and if you wrote a custom data structure (say, a PatientQueue), there was no standard contract for how to walk through it.

Iterable and Iterator standardize traversal: any class that implements Iterable can be used in a for-each loop, regardless of its internal structure (array, linked list, tree, hash table).

Section Overview

- Iterable<T> — the interface

```
> public interface Iterable<T> {
```

- Iterator<T> iterator();
- }
- Iterator<T> — the interface

```
> public interface Iterator<T> {
```

- boolean hasNext();
- T next();
- default void remove() { throw new UnsupportedOperationException(); }
- }

Section Overview

- Three methods under iterator

Section Overview

- Vector<String> patients = new Vector<>();
- patients.add("Ravi"); patients.add("Priya"); patients.add("Kumar");
- Iterator<String> it = patients.iterator(); // patients (Iterable) produces an Iterator
- while (it.hasNext()) {
- String name = it.next();

```
> System.out.println(name);
```

- }

Section Overview

Using iterator we can remove elements while traversing

```
> // CORRECT — use Iterator's own remove()
```

- Iterator<String> it = patients.iterator();
- while (it.hasNext()) {
- String name = it.next();
- if (name.equals("Priya")) {
- it.remove(); // safe — Iterator knows how to fix its internal cursor
- }
- }

Java Math Class

The java.lang.Math class contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.

- This class inherits methods from the following classes –
- java.lang.Object

Section Overview

```
> public class Main {
> public static void main(String[] args) {
> System.out.println(Math.abs(-10)); // 10
> System.out.println(Math.sqrt(25)); // 5.0
> System.out.println(Math.pow(2, 3)); // 8.0
> System.out.println(Math.max(10, 20)); // 20
> System.out.println(Math.min(10, 20)); // 10
> System.out.println(Math.ceil(4.3)); // 5.0
> System.out.println(Math.floor(4.8)); // 4.0
> System.out.println(Math.round(4.6)); // 5
> System.out.println(Math.random()); // 0.0 to < 1.0
```

- }
- }